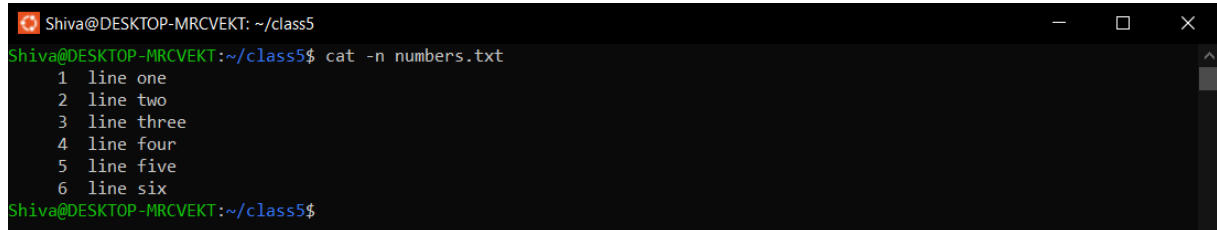


Linux Class 5 - Task

Picking Lines from a File with head and tail

For practice I made a folder called class5 and inside it a file called numbers.txt with six lines, using vi. To check my answers I first looked at the file with cat -n, which puts the line number in front of every line.



```
Shiva@DESKTOP-MRCVEKT: ~/class5
Shiva@DESKTOP-MRCVEKT:~/class5$ cat -n numbers.txt
 1 line one
 2 line two
 3 line three
 4 line four
 5 line five
 6 line six
Shiva@DESKTOP-MRCVEKT:~/class5$
```

Screenshot 1: The practice file, shown with line numbers using cat -n.

Two commands are used in this task. head shows lines counting from the top of the file, and tail shows lines counting from the bottom. The number after -n says how many lines.

1. How to fetch only the first and last line of a file using head and tail?

The first line comes from head and the last line comes from tail:

```
head -1 numbers.txt
```

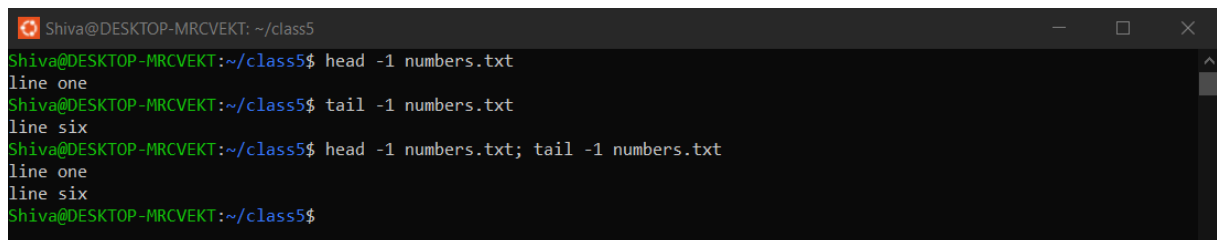
```
tail -1 numbers.txt
```

To get both at once I put them on one line with a semicolon in between:

```
head -1 numbers.txt; tail -1 numbers.txt
```

The semicolon simply means run this command, and when it finishes run the next one. So head prints line one, then tail prints line six.

- -1 means one line. head -n 1 is the same thing written in the full way.
- head goes from the top, tail goes from the bottom. That is the only difference.



```
Shiva@DESKTOP-MRCVEKT: ~/class5
Shiva@DESKTOP-MRCVEKT:~/class5$ head -1 numbers.txt
line one
Shiva@DESKTOP-MRCVEKT:~/class5$ tail -1 numbers.txt
line six
Shiva@DESKTOP-MRCVEKT:~/class5$ head -1 numbers.txt; tail -1 numbers.txt
line one
line six
Shiva@DESKTOP-MRCVEKT:~/class5$
```

Screenshot 2: head and tail separately, and then both together with a semicolon.

2. How do I print all lines of a file except the 4th line?

There is no option that says skip line 4. So I printed the lines before it and then the lines after it:

```
head -3 numbers.txt
```

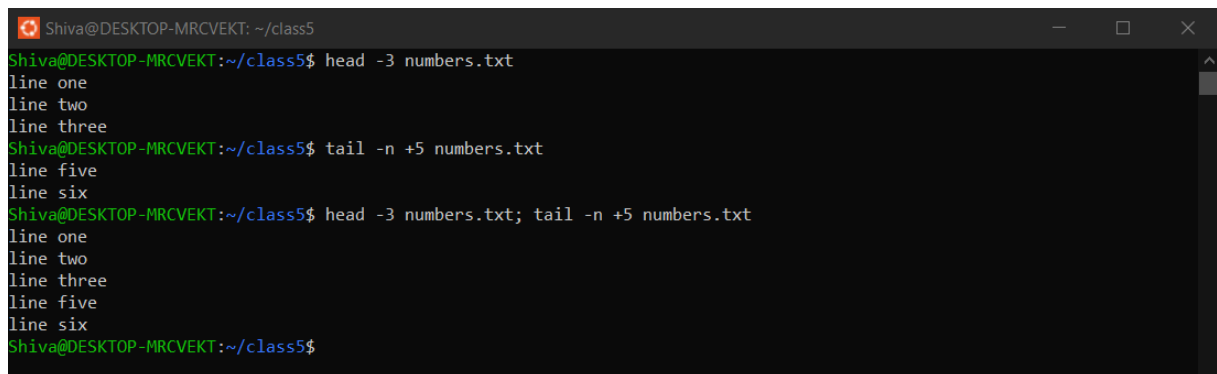
That gives lines 1, 2 and 3. Then I need line 5 onwards:

```
tail -n +5 numbers.txt
```

Putting them together prints the whole file with the 4th line missing:

```
head -3 numbers.txt; tail -n +5 numbers.txt
```

The important part here is the plus sign. `tail -n +5` means start from line 5 and go to the end. Without the plus, `tail -n 5` means the last 5 lines, which is a completely different answer. This is the mistake I made the first time.



```
Shiva@DESKTOP-MRCVEKT: ~/class5
Shiva@DESKTOP-MRCVEKT:~/class5$ head -3 numbers.txt
line one
line two
line three
Shiva@DESKTOP-MRCVEKT:~/class5$ tail -n +5 numbers.txt
line five
line six
Shiva@DESKTOP-MRCVEKT:~/class5$ head -3 numbers.txt; tail -n +5 numbers.txt
line one
line two
line three
line five
line six
Shiva@DESKTOP-MRCVEKT:~/class5$
```

Screenshot 3: line four is missing from the output. The rest of the file is there.

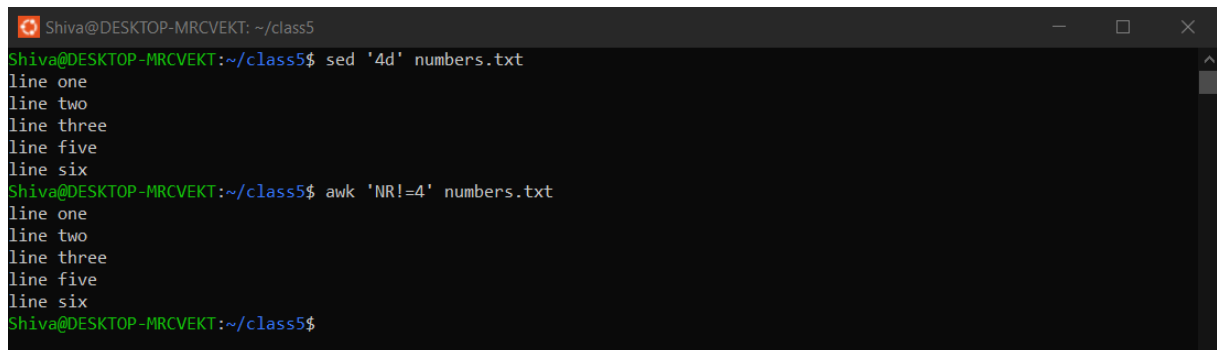
There are two shorter ways to do the same thing:

```
sed '4d' numbers.txt
```

Here `d` means delete, so it prints the file without line 4. It only changes what is shown on the screen, not the file itself.

```
awk 'NR!=4' numbers.txt
```

`NR` is the line number that `awk` is on, and `!=` means not equal. So it prints every line where the line number is not 4.



```
Shiva@DESKTOP-MRCVEKT: ~/class5
Shiva@DESKTOP-MRCVEKT:~/class5$ sed '4d' numbers.txt
line one
line two
line three
line five
line six
Shiva@DESKTOP-MRCVEKT:~/class5$ awk 'NR!=4' numbers.txt
line one
line two
line three
line five
line six
Shiva@DESKTOP-MRCVEKT:~/class5$
```

Screenshot 4: the same answer using `sed` and using `awk`.

Commands I used

Command	What it does
<code>head -1 file</code>	Shows the first line.
<code>tail -1 file</code>	Shows the last line.
<code>head -1 f; tail -1 f</code>	First and last line together. The <code>;</code> runs one command after the other.
<code>head -3 file</code>	The first 3 lines.
<code>tail -n +5 file</code>	From line 5 to the end. The <code>+</code> is what makes it count from the top.
<code>tail -n 5 file</code>	The last 5 lines. Not the same as the one above.
<code>sed '4d' file</code>	Prints the file without line 4. The file itself is not changed.

Command	What it does
<code>awk 'NR!=4' file</code>	Same result. NR is the line number.
<code>cat -n file</code>	Shows the file with line numbers, to check the answers.
<code>wc -l file</code>	Counts how many lines the file has.

If the file is long I can use `wc -l numbers.txt` first to see how many lines there are, so I know what number to give to head and tail.